

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ

«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

«Алгоритмы и структуры данных»

отчет по блоку задач №1

Выполнил:

Студент группы Р3209

Поляков А. Л.

Санкт-Петербург, 2026г.

Задача А. Агроном-любитель

Описание:

Городской школьник Лёша поехал на лето в деревню и занялся выращиванием цветов. Он посадил n цветков вдоль одной длинной прямой грядки, и они успешно выросли. Лёша посадил множество различных видов цветков, i -й от начала грядки цветок имеет вид a_i , где a_i — целое число, номер соответствующего вида в «Каталоге юного агронома».

Теперь Лёша хочет сделать фотографию выращенных им цветов и выложить ее в раздел «мои грядки» в социальной сети для агрономов «ВКомпосте». На фотографии будет виден отрезок из одного или нескольких высаженных подряд цветков.

Однако он заметил, что фотография смотрится не очень интересно, если на ней много одинаковых цветков подряд. Лёша решил, что если на фотографии будут видны три цветка одного вида, высаженные подряд, то его друзья — специалисты по эстетике цветочных фотографий — поставят мало лайков.

Помогите ему выбрать для фотографирования как можно более длинный участок своей грядки, на котором нет трех цветков одного вида подряд.

Формат ввода

В первой строке содержится целое число n ($1 \leq n \leq 200\,000$) — количество цветов на грядке.

Во второй строке содержится n целых чисел a_i ($1 \leq a_i \leq 10^9$), обозначающих вид очередного цветка. Одинаковые цветки обозначаются одинаковыми числами, разные — разными.

Формат вывода

Выведите номер первого и последнего цветка на самом длинном искомом участке. Цветки нумеруются от 1 до n .

Если самых длинных участков несколько, выведите участок, который начинается раньше.

Решение:

```
#include <iostream>
#include <string>
#include <vector>

using namespace std;

int main() {
    int last = 0, maxlen = 0, left = 0, right = 0;
    int n = 0;
    cin >> n;
    vector<int> s = vector<int>(n);
    for (int i = 0; i < n; i++) {
        cin >> s[i];
    }

    for (int i = 0; i < n - 2; i++) {
        if (s[i] == s[i + 1] && s[i] == s[i + 2]) {
            if (i + 1 - last > maxlen) {
                maxlen = i + 1 - last;
                left = last, right = i + 1;
            }
            last = i + 1;
        }
    }
    if (n - 1 - last > maxlen) {
        cout << last + 1 << " " << n;
    } else {
        cout << left + 1 << " " << right + 1;
    }
}
```

```
}  
    return 0;  
}
```

Объяснение решения:

По сути, нам просто надо идти по массиву, пока мы не встретим 3 подряд одинаковых элементов. Поэтому просто идем по массиву, сохраняя максимальную длину подходящего подмассива, а также сохраняя индексы его начала и конца.

Время: $O(n)$

Память: $O(n)$

Задача В. Зоопарк Глеба

Описание:

Недавно Глеб открыл зоопарк. Он решил построить его в форме круга и, естественно, обнёс забором. Глеб взял вас туда начальником охраны. Казалось бы все началось так хорошо, но именно в вашу первую смену все животные разбежались. В зоопарке n животных различных видов, также под каждый из видов есть свои ловушки. К сожалению некоторые животные враждуют с друг другом в природе (они обозначены разными буквами), а зоопарк обнесён забором и имеет форму круга. С помощью камер, удалось выяснить, где находятся все животные. Умная система поддержки жизнедеятельности зоопарка уже просканировала зоопарк и вывела id всех животных и ловушек в том порядке, в котором они видны из центра зоопарка. Получилось так, что все животные и все ловушки находятся на краю зоопарка. Вы хотите понять, могут ли животные прийти в свою ловушку так, чтобы их путь не пересекался с другими. Если да, также предьявите какую-нибудь из схем поимки животных.

Формат ввода

На вход подается строка из $2 \cdot n$ символов латинского алфавита, где маленькая буква - животное, а большая - ловушка. Размер строки не более 100000.

Формат вывода

Требуется вывести "Impossible", если решения не существует или "Possible", если можно вернуть всех животных в клетки. В случае если можно, то для каждой ловушки в порядке обхода требуется вывести индекс животного в ней.

Решение:

```
#include <iostream>  
#include <stack>  
#include <vector>  
  
using namespace std;  
  
int main() {  
    string stroke;  
    cin >> stroke;  
    stack<pair<char, int>> stck;  
    int animal_counter = 1;  
    int trap_counter = 0;  
    vector<int> answers(stroke.size() / 2, 0);
```

```

for (char c : stroke) {
    if ((stck.size() == 0) || (tolower(stck.top().first) != tolower(c))
||
        (bool(islower(c)) == bool(islower(stck.top().first)))) {
        if (islower(c)) {
            stck.push(make_pair(c, animal_counter));
            animal_counter++;
        } else {
            stck.push(make_pair(c, trap_counter));
            trap_counter++;
        }
        } else {
            if (islower(c)) {
                answers[stck.top().second] = animal_counter;
                animal_counter++;
                stck.pop();
            } else {
                answers[trap_counter] = stck.top().second;
                trap_counter++;
                stck.pop();
            }
        }
    }
    if (stck.size() == 0) {
        cout << "Possible" << "\n";
        for (auto i : answers) {
            cout << i << ' ';
        }
    } else {
        cout << "Impossible";
    }
    return 0;
}

```

Объяснение Решения:

Достаточно сложная мысль, но спустя какое-то время размышлений над задачей она приходит. Условие про нахождение ловушек и животных на окружности очень похоже на условие правильной скобочной последовательности. Единственное что, у нас нет открывающей и закрывающей скобки, есть животное и ловушка. Поэтому можно решить задачу так же, как и задачу на скобочную последовательность: с помощью стека. Ну и так как нам нужно будет потом вывести какое животное в какую ловушку попало, будем сохранять не просто букву, а и ее индекс: отдельный ряд индексов для животных и отдельный для ловушек. При подходящей паре в вектор answer по индексу ловушки кладем индекс животного, которое туда попало. Потом если стек к концу строки не пуст, значит там осталось животное/ловушка без пары, следовательно ответ будет Impossible. Иначе выводим вектор answer.

Время: $O(n)$

Память: $O(n)$

Задача C. Конфигурационный файл

Описание:

Вадим разрабатывает парсер конфигурационных файлов для своего проекта. Файл состоит из блоков, которые выделяются с помощью символов «{» — начало блока, и «}» — конец блока. Блоки могут вкладываться друг в друга. В один блок может быть вложено несколько других блоков.

В конфигурационном файле встречаются переменные. Каждая переменная имеет имя, которое состоит из не более чем десяти строчных букв латинского алфавита. Переменным можно присваивать числовые значения. Изначально все переменные имеют значение 0.

Присваивание нового значения записывается как `<variable>=<number>`, где `<variable>` — имя переменной, а `<number>` — целое число, по модулю не превосходящее 10^9 . Парсер читает конфигурационный файл построчно. Как только он встречает выражение присваивания, он присваивает новое значение переменной. Это значение сохраняется до конца текущего блока, а затем восстанавливается старое значение переменной. Если в блок вложены другие блоки, то внутри тех из них, которые идут после присваивания, значение переменной также будет новым.

Кроме того, в конфигурационном файле можно присваивать переменной значение другой переменной. Это действие записывается как `<variable1>=<variable2>`. Прочитав такую строку, парсер присваивает текущее значение переменной `variable2` переменной `variable1`. Как и в случае присваивания константного значения, новое значение сохраняется только до конца текущего блока. После окончания блока переменной возвращается значение, которое было перед началом блока.

Для отладки Вадим хочет напечатать присваиваемое значение для каждой строки вида `<variable1>=<variable2>`. Помогите ему отладить парсер.

Формат ввода

Входные данные содержат хотя бы одну и не более 10^5 строк. Каждая строка имеет один из четырех типов:

- { — начало блока;
- } — конец блока;
- `<variable>=<number>` — присваивание переменной значения, заданного числом;
- `<variable1>=<variable2>` — присваивание одной переменной значения другой переменной. Переменные `<variable1>` и `<variable2>` могут совпадать.

Гарантируется, что ввод является корректным и соответствует описанию из условия. Ввод не содержит пробелов.

Формат вывода

Для каждой строки типа `<variable1>=<variable2>` выведите значение, которое было присвоено.

Решение:

```
#include <iostream>
#include <map>
#include <ostream>
#include <stack>

using namespace std;

int main() {
    string stroke;
    map<string, int> val;
    stack<pair<string, int>> history;

    while (cin >> stroke) {
        if (stroke == "{") {
            history.push(make_pair("{", 0));
```

```

    } else if (stroke == "{") {
        while (history.top().first != "{") {
            val[history.top().first] = history.top().second;
            history.pop();
        }
        history.pop();
    } else {
        string a, b;
        int x;
        x = stroke.find("=");
        a = stroke.substr(0, x);
        b = stroke.substr(x + 1);
        history.push(make_pair(a, val[a]));
        if (b[0] == '-' || isdigit(b[0])) {
            val[a] = stoi(b);
        } else if (val.contains(b)) {
            val[a] = val[b];
            cout << val[a] << endl;
        } else {
            val[a] = 0;
            cout << 0 << endl;
        }
    }
}
}
}

```

Объяснение Решения:

Создаем словарь, в котором храним уже заданные переменные и их значения, а также логгер в виде стека, в котором храним все старые значение измененных переменных, которые происходили с переменными. Если у нас строка "{" записываем ее в логгер. Это будет индикатор, до которого мы будем откатываться при выходе из блока. Когда встречаем строку закрытия блока ("}"), идем по логгеру и присваиваем переменным старые значения до тех пор, пока не встретим "{". Вывод значения переменной при присваивании работает максимально просто. Делим строку по знаку «=», если справа у нас число, то просто в словаре присваиваем переменной, которая записана в первой части строки значение из второй части. Если слева не число, проверяем, есть ли переменная в стеке, если нет, то присваиваем переменной в первой части 0 и выводим его. Иначе присваиваем переменной слева значение переменной справа и, опять же, выводим его.

Время $O(n)$

Память $O(n)$

Задача D. Профессор Хаос

Описание:

В секретной лаборатории профессора Хаоса проходит эксперимент по выращиванию особо опасных бактерий. В начале первого дня эксперимента у Хаоса имеется a особо опасных бактерий.

Каждый день эксперимента устроен следующим образом. Рано утром профессор достает из контейнера все свои бактерии и помещает их в инкубатор, где бактерии начинают делиться. Вместо каждой бактерии образуется b новых бактерий.

После извлечения бактерий из инкубатора с из них используются для проведения различных опытов и затем уничтожаются. Если после извлечения из инкубатора имеется менее c бактерий, для проведения опытов используются все имеющиеся бактерии, и эксперимент заканчивается.

Оставшиеся бактерии в конце дня необходимо поместить в контейнер и продолжить использовать в эксперименте. Однако в контейнер можно поместить не более d бактерий, поэтому если число оставшихся бактерий больше d , то в контейнер помещаются d бактерий, а остальные уничтожаются.

Теперь профессор Хаос хочет выяснить, сколько особо опасных бактерий будет у него в контейнере после k -го дня эксперимента. Помогите ему найти ответ на этот вопрос.

Формат ввода

В единственной строке входного файла содержится пять целых чисел a, b, c, d и k ($1 \leq a, b \leq 1000, 0 \leq c \leq 1000, 1 \leq d \leq 1000, a \leq d, 1 \leq k \leq 10^{18}$).

Формат вывода

Выведите одно число — количество бактерий у Хаоса к концу k -го дня. Если эксперимент завершится в k -й день или ранее, выведите число 0.

Решение:

```
#include <iostream>

using namespace std;

int main() {
    int a, b, c, d, k, last;
    cin >> a >> b >> c >> d >> k;
    for (int i = 1; i <= k; i++) {
        last = a;
        a = min(a * b - c, d);
        if (a <= 0) {
            cout << 0;
            return 0;
        }
        if (a == last) {
            cout << a;
            return 0;
        }
    }
    cout << a;
    return 0;
}
```

Описание решения:

Сначала просто переписываем условие в код, и запускаем. Работает, но выходит за лимит по времени. Первая мысль – а либо монотонно убывает, либо монотонно возрастает, либо же не изменяется. Доказать это легко: если мы будем подставлять в формулу $a*b-c$ переменную a больше или меньше изначальной, то значение будет точно так же больше или меньше, чем изначальное, т.к. переменная a здесь просто множитель. Тогда у нас всего 4 исхода: либо a станет равна 0, либо не будет изменяться, либо упрется в лимит d , или циклы закончатся. Нас интересует случай, когда k слишком большое, поэтому последний пункт мы не рассматриваем. Тогда можно просто создать условие, что мы будем сравнивать a с a на прошлом цикле, и, если они равны, просто выходить. Тогда в худшем случае, программа завершится за d циклов. А на d ограничение гораздо приятнее: $1 \leq d \leq 1000$.

Время: $O(\min(d, k))$

Память: $O(1)$

Задачи Timus

Задача №1296

Описание:

Гиперпереход, открытый ещё в начале XXI-го века, и сейчас остаётся основным способом перемещения на расстояния до сотен тысяч парсеков. Но совсем недавно физиками открыто новое явление. Оказывается, длительностью альфа-фазы перехода можно легко управлять. Корабль, находящийся в альфа-фазе перехода, накапливает гравитационный потенциал. Чем больше накопленный гравитационный потенциал корабля, тем меньше энергии потребуется ему на прыжок сквозь пространство. Ваша цель — написать программу, которая позволит кораблю за счёт выбора времени начала альфа-фазы и её длительности накопить максимальный гравитационный потенциал.

В самой грубой модели грави-интенсивность — это последовательность целых чисел p_i . Будем считать, что если альфа-фаза началась в момент i и закончилась в момент j , то накопленный в течение альфа-фазы потенциал — это сумма всех чисел, стоящих в последовательности на местах от i до j .

Исходные данные

В первой строке записано целое число N — длина последовательности, отвечающей за грави-интенсивность ($0 \leq N \leq 60000$). Далее идут N строк, в каждой записано целое число p_i ($-30000 \leq p_i \leq 30000$).

Результат

Выведите максимальный гравитационный потенциал, который может накопить корабль в альфа-фазе прыжка. Считается, что потенциал корабля в начальный момент времени равен нулю.

Решение:

```
#include "iostream"
#include "algorithm"
#include "string"

using namespace std;

int main() {
    int n;
    int answer = 0;
    int cur = 0;
    cin >> n;
    for (int i = 0; i < n; i++) {
        string input;
        cin >> input;
        int el = stoi(input);
        cur = max(cur + el, 0);
        answer = max(answer, cur);
    }
    cout << answer;
    return 0;
}
```

Объяснение решения:

Задача на подмассив максимальной суммы. Идем по массиву, складываем элементы.

Сохраняем максимальную сумму подмассива. Если Сумма подмассива меньше нуля, то начинаем новый подмассив, т. к. часть слева только уменьшает сумму, а нам нужно найти максимальную.

Задача №1401

Описание:

Известно, что господин Чичиков зарабатывал свой капитал и таким способом: он спорил со всякими недотёпами, что сможет доказать, что квадратную доску размера 512×512 нельзя замостить следующими фигурами:

x

xx

x

xx

xx

x

xx

x

и всегда выигрывал. Однако один из недотёп оказался не так уж глуп, и сказал, что сможет замостить такими фигурами доску размера 512×512 без правой верхней клетки. Чичиков, не подумав, ляпнул, что он вообще может любую доску размера $2^n \times 2^n$ без одной произвольной клетки замостить такими фигурами. Слово за слово, они поспорили. Чичиков чувствует, что сам он не докажет свою правоту. Помогите же ему!

Исходные данные

В первой строке записано целое число n ($1 \leq n \leq 9$). Во второй строке через пробел даны два целых числа x, y : координаты «выколотой» клетки доски ($1 \leq x, y \leq 2^n$), x — номер строки, y — номер столбца. Левый верхний угол доски имеет координаты $(1, 1)$.

Результат

Ваша программа должна выдать 2^n строчек по 2^n чисел в каждой строке. На месте выбитой клетки должно стоять число 0. На месте остальных клеток должны стоять числа от 1 до $(2^{2n} - 1) / 3$ — номер фигуры, закрывающей данную клетку. Разумеется, одинаковые номера должны образовывать фигуры. Если же такую доску нельзя покрыть фигурами, выведите «-1».

Пример

исходные данные	результат
2 1 1	0 1 3 3 1 1 4 3 2 4 4 5 2 2 5 5

Решение:

```
#include <iostream>
#include <vector>

using namespace std;

vector<vector<int>> field;
int point_x, point_y;
int counter = 1;

bool check(int start_x, int start_y, int size) {
    return (
        (start_x <= point_x && point_x <= start_x + size - 1) &&
        (start_y <= point_y && point_y <= start_y + size - 1)
    );
}

void block(int start_x, int start_y, int size) {
    if (size == 2) {
        if (check(start_x, start_y, 1) || field[start_x][start_y] != 0) {
            field[start_x + 1][start_y + 1] = counter;
            field[start_x][start_y + 1] = counter;
            field[start_x + 1][start_y] = counter;
            counter++;
            return;
        } else if (check(start_x + 1, start_y, 1) || field[start_x + 1][start_y]
!= 0) {
```

```

        field[start_x + 1][start_y + 1] = counter;
        field[start_x][start_y + 1] = counter;
        field[start_x][start_y] = counter;
        counter++;
        return;
    } else if (check(start_x, start_y + 1, 1) || field[start_x][start_y + 1]
!= 0) {
        field[start_x + 1][start_y + 1] = counter;
        field[start_x + 1][start_y] = counter;
        field[start_x][start_y] = counter;
        counter++;
        return;
    } else if (check(start_x + 1, start_y + 1, 1) || field[start_x +
1][start_y + 1] != 0) {
        field[start_x][start_y + 1] = counter;
        field[start_x + 1][start_y] = counter;
        field[start_x][start_y] = counter;
        counter++;
        return;
    }
    return;
}
int half = size / 2;
if (check(start_x, start_y, half) || field[start_x][start_y] != 0) {
    field[start_x + half][start_y + half] = counter;
    field[start_x + half - 1][start_y + half] = counter;
    field[start_x + half][start_y + half - 1] = counter;
} else if (check(start_x + half, start_y, half) || field[start_x + size -
1][start_y] != 0) {
    field[start_x + half][start_y + half] = counter;
    field[start_x + half - 1][start_y + half] = counter;
    field[start_x + half - 1][start_y + half - 1] = counter;
} else if (check(start_x, start_y + half, half) || field[start_x][start_y +
size - 1] != 0) {
    field[start_x + half][start_y + half] = counter;
    field[start_x + half][start_y + half - 1] = counter;
    field[start_x + half - 1][start_y + half - 1] = counter;
} else if (check(start_x + half, start_y + half, half) ||
        field[start_x + size - 1][start_y + size - 1] != 0) {
    field[start_x + half - 1][start_y + half - 1] = counter;
    field[start_x + half][start_y + half - 1] = counter;
    field[start_x + half - 1][start_y + half] = counter;
}
counter++;
block(start_x, start_y, half);
block(start_x + half, start_y, half);
block(start_x, start_y + half, half);
block(start_x + half, start_y + half, half);
}

void print_filed() {
    for (auto i : field) {
        for (auto j : i) {
            cout << j << " ";
        }
        cout << endl;
    }
}

int main() {
    int n;
    cin >> n >> point_x >> point_y;
    point_x--;
    point_y--;

```

```
n = pow(2, n);  
field.resize(n, vector<int>(n, 0));  
block(0, 0, n);  
print_filed();  
return 0;  
}
```

Объяснение решения:

Сначала докажем, что условие можно выполнить всегда. Докажем по индукции.

База: для поля размером 2×2 условие выполняется, это очевидно.

Индукционное предположение: если выполняется для поля $2^n \times 2^n$, то для поля $2^{n+1} \times 2^{n+1}$ тоже будет выполняться.

Доказательство: располагаем фигуру в 4 центральных клетках так, чтобы в четверти где находится выколотое поле фигура не лежала. Разбиваем поле $2^{n+1} \times 2^{n+1}$ на 4 одинаковых поля размером в 2 раза меньше: $2^n \times 2^n$. Получается, в каждом поле $2^n \times 2^n$ есть одна уже занятая точка: либо выколотая, либо часть фигуры, которую мы положили. А поле $2^n \times 2^n$ мы можем замостить. Индукция доказана.

Теперь осталось записать этот метод кодом. Создаем поле нужного размера, если у нас есть минимальное неделимое поле 2×2 просто располагаем фигуру правильным образом и заканчиваем. В ином случае смотрим, где находится выколотая точка, правильно располагаем центральную фигуру и делим поле на 4 поля в 2 раза меньше, рекурсивно вызывая функцию.

Время $O(4^n)$

Память $O(4^n)$